STAT-S 650 Time Series Analysis

Homework 6

Manish Suresh Maudgalya

Theory

1. Put the following ARMA model in state space form ($w(t)$ is white noise, and $x(t)$ is a known input)

$$y(t) = \beta_0 + \beta_1 \cdot x(t-1) + \alpha_1 \cdot y(t-1) + \alpha_2 \cdot y(t-2) + w(t) + \theta_1 \cdot w(t-1) + \theta_2 \cdot w(t-2)$$

2. Is the above model stable if we set $\alpha_1 = .95$ and $\alpha_2 = .35$? What would the model predict in this case?

```r
#2. State-space representation
# State vector:
# s_t = [ y_t, y_{t-1}, w_t, w_{t-1} ]'
#
# s_t   = A s_{t-1} + B u_t + K * e_t   (here we just use w_t as noise)
# y_t   = C s_t + D u_t + w_t
#
# We encode the ARMA structure in A, B, C, D.

A <- matrix(c(
  alpha1, alpha2, theta1, theta2,
  1,       0,      0,      0,
  0,       0,      0,      0,
  0,       0,      1,      0
), nrow = 4, byrow = TRUE)

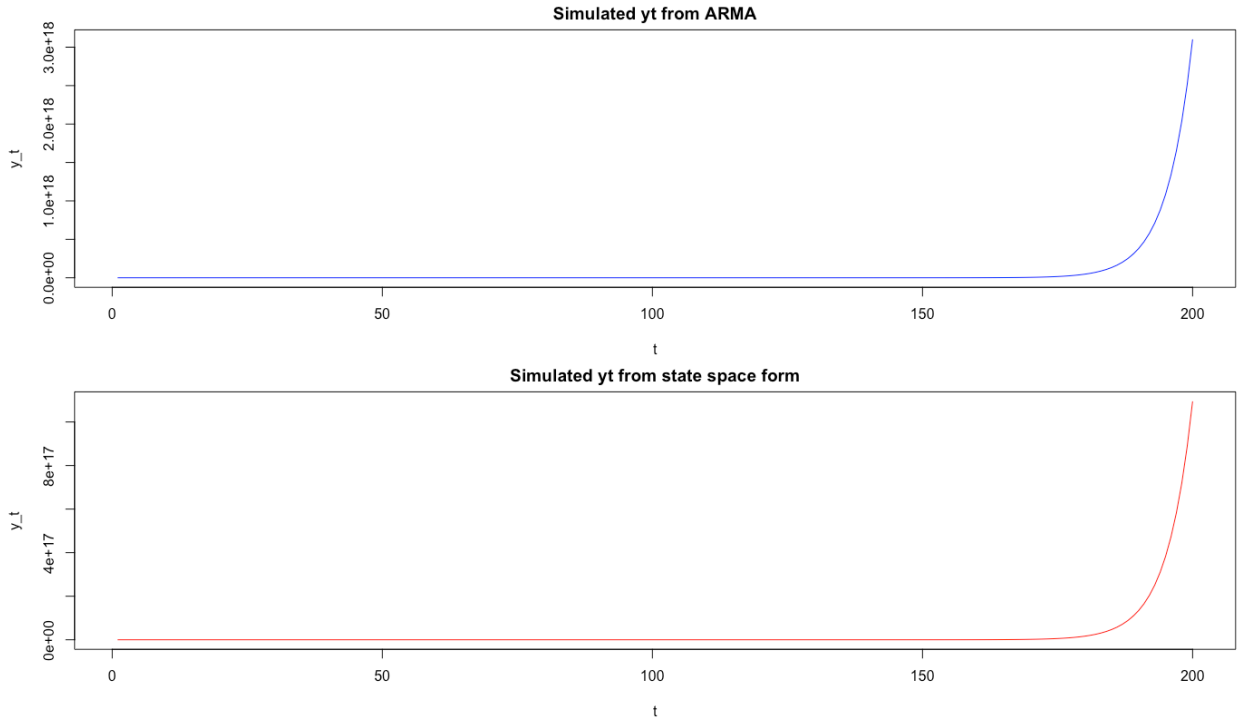
B <- matrix(c(beta1, 0, 0, 0), nrow = 4, byrow = TRUE)

C <- matrix(c(1, 0, 1, 0), nrow = 1)  # y_t depends on y_t and w_t in state
D <- beta0                            # intercept term

cat("A matrix:\n"); print(A)
cat("B matrix:\n"); print(B)
cat("C matrix:\n"); print(C)
cat("D (intercept):\n"); print(D)
```

```
AR(2) roots:
[1]  0.8105701+4.762511e-16i -3.5248558-4.762511e-16i
> cat("Moduli of roots:\n"); print(Mod(roots))
Moduli of roots:
[1] 0.8105701 3.5248558
>
> if (all(Mod(roots) > 1)) {
+   cat("\nThe AR(2) part is STABLE\n")
+ } else {
+   cat("\nThe AR(2) part is UNSTABLE\n")
+ }

The AR(2) part is UNSTABLE
```



As we can see the explosion upward, the Model is UNSTABLE.

Practice

3. Pick at least one output variable and at least one input variable from your data set. Formulate different noise models using combinations of AR and MA components. If needed include a seasonal model.
4. Fit the models using the ARMAX model.
5. Compare the models using BIC and AIC and Chi square (if nested)
6. Choose the best model of those you compared. Justify your choice.
7. Examine the residuals of the model to see if they are white noise using autocovariance on the residuals and specral analysis of the residuals.
8. Prepare an interpretation of the parameters of the model.
9. Plot the predictions of the model using hypothetical inputs with an R program.

Models Used:

```r
X <- matrix(nasdaq_ret, ncol = 1)   # exogenous regressor matrix

fit_armax <- function(p, q, xreg = X) {
  Arima(sp500_ret, order = c(p, 0, q), xreg = xreg, include.mean = TRUE)
}

cat(" Fitting ARMAX Models \n")
M1 <- fit_armax(1, 0)  # AR(1)
M2 <- fit_armax(0, 1)  # MA(1)
M3 <- fit_armax(1, 1)  # ARMA(1,1)
M4 <- fit_armax(2, 1)  # ARMA(2,1)
M5 <- fit_armax(1, 2)  # ARMA(1,2)
M6 <- fit_armax(2, 2)  # ARMA(2,2)

models      <- list(M1 = M1, M2 = M2, M3 = M3, M4 = M4, M5 = M5, M6 = M6)
model_names <- c("ARMAX(1,0)", "ARMAX(0,1)", "ARMAX(1,1)",
                 "ARMAX(2,1)", "ARMAX(1,2)", "ARMAX(2,2)")
```

AIC and BIC values for the Models

```
> comparison <- comparison[order(comparison$BIC), ]
> cat("\n Model Comparison (sorted by BIC) \n")

 Model Comparison (sorted by BIC)
> print(comparison)
        Model       AIC       BIC   LogLik nParams
M1 ARMAX(1,0) -34352.73 -34327.53 17180.37       4
M2 ARMAX(0,1) -34352.66 -34327.46 17180.33       4
M3 ARMAX(1,1) -34350.70 -34319.20 17180.35       5
M5 ARMAX(1,2) -34351.06 -34313.27 17181.53       6
M4 ARMAX(2,1) -34350.89 -34313.10 17181.45       6
M6 ARMAX(2,2) -34351.52 -34307.42 17182.76       7
> lrt <- function(small, large, label) {
+   stat <- 2 * (as.numeric(logLik(large)) - as.numeric(logLik(small)))
+   df   <- length(coef(large)) - length(coef(small))
+   pval <- pchisq(stat, df = df, lower.tail = FALSE)
```

CHI Squared Test for the Nested Models

```
>
> cat("\n Likelihood Ratio (Chi-square) Tests for Nested Models \n")

 Likelihood Ratio (Chi-square) Tests for Nested Models
> lrt(M1, M3, "M1 vs M3 (+MA1)")
  M1 vs M3 (+MA1)       chi2=-0.029  df=1  p=1.0000
> lrt(M2, M3, "M2 vs M3 (+AR1)")
  M2 vs M3 (+AR1)       chi2=0.044  df=1  p=0.8342
> lrt(M3, M4, "M3 vs M4 (+AR2)")
  M3 vs M4 (+AR2)       chi2=2.191  df=1  p=0.1389
> lrt(M3, M5, "M3 vs M5 (+MA2)")
  M3 vs M5 (+MA2)       chi2=2.360  df=1  p=0.1245
> lrt(M4, M6, "M4 vs M6 (+MA2)")
  M4 vs M6 (+MA2)       chi2=2.622  df=1  p=0.1054
> lrt(M5, M6, "M5 vs M6 (+AR2)")
  M5 vs M6 (+AR2)       chi2=2.452  df=1  p=0.1173
> #
```

BEST Model:

Here I am using the ARMAX(1,0) model. BIC is the most reliable criterion here because it penalizes complexity more heavily than AIC, which matters when differences between models are small. M1 has the lowest BIC of -34327.53. AIC prefers M1 (−34352.73)

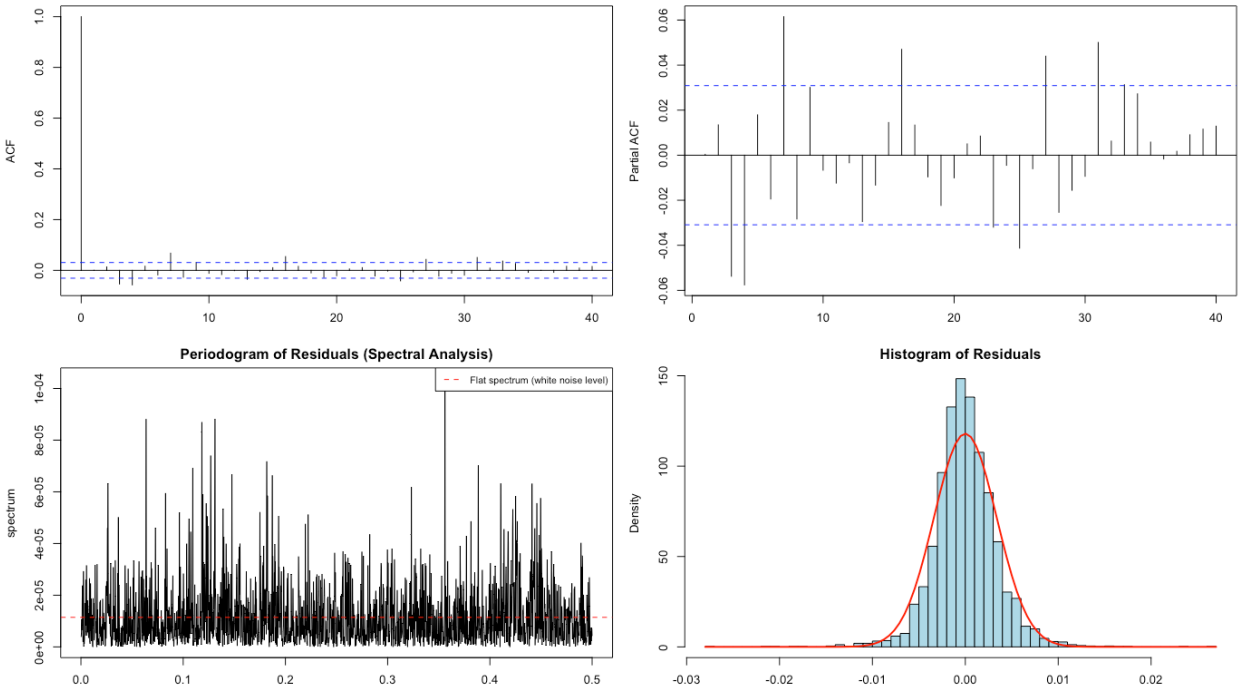
```
Series: sp500_ret
Regression with ARIMA(1,0,0) errors

Coefficients:
          ar1  intercept    xreg
      -0.0261      0e+00  0.8029
s.e.   0.0158      1e-04  0.0041

sigma^2 = 1.144e-05:  log likelihood = 17180.37
AIC=-34352.73   AICc=-34352.72   BIC=-34327.53

Training set error measures:
                     ME        RMSE         MAE  MPE MAPE      MASE         ACF1
Training set 7.74599e-07 0.003381369 0.002455111 -Inf  Inf 0.2298902 0.0003852623
> #
> #  Residual checking white noise
```

Periodogram, ACF, PACF and Histogram of the Residuals.



Interpretation of the Parameters of the Model:

	Parameter	Estimate	Std_Error	t_value	Signif
1	ar1	-0.026070	0.015823	-1.648	*
2	intercept	-0.000015	0.000054	-0.268	
3	xreg	0.802909	0.004142	193.856	***

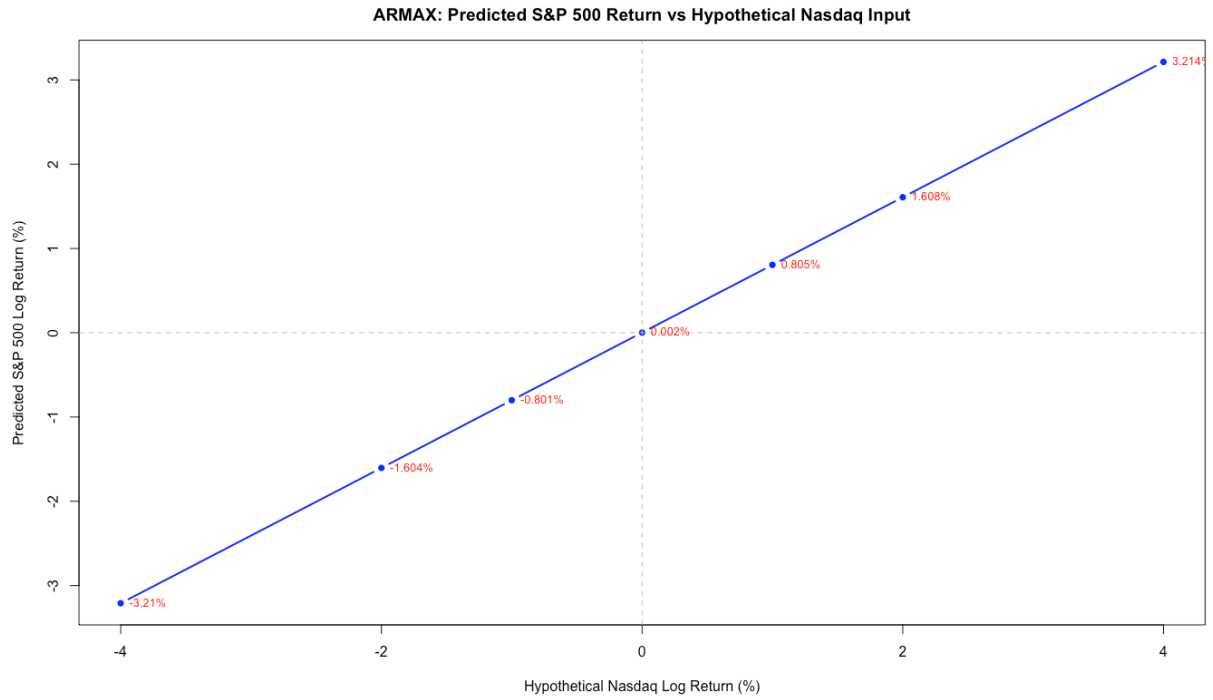
XREG coefficient: elasticity of S&P 500 log return to Nasdaq log return. A coefficient near 1 means the two indices move almost one-for-one.

AR coefficients (ar1, ar2): describe autocorrelation in the noise after removing the exogenous Nasdaq effect.

MA coefficients (ma1, ma2): capture moving-average structure in the noise; reflect how shocks to S&P 500 beyond what Nasdaq explains persist over 1-2 days.

Intercept / mean: average daily S&P 500 log return not explained by Nasdaq.

Prediction Plot:



R Code

```r
# ARMA in state-space form
# Model
# y_t = beta0 + beta1 * x_{t-1} + alpha1 * y_{t-1} + alpha2 * y_{t-2} + w_t + theta1 * w_{t-1} + theta2 * w_{t-2}

rm(list = ls())

#1. Parameters
beta0  <- 0.5
beta1  <- 1.2
alpha1 <- 0.95
alpha2 <- 0.35
theta1 <- 0.4
theta2 <- -0.2



#2. State-space representation
# State vector:
# s_t = [ y_t, y_{t-1}, w_t, w_{t-1} ]'
#
# s_t   = A s_{t-1} + B u_t + K * e_t   (here we just use w_t as noise)
# y_t   = C s_t + D u_t + w_t
#
# We encode the ARMA structure in A, B, C, D.

A <- matrix(c(
  alpha1, alpha2, theta1, theta2,
  1,       0,      0,      0,
  0,       0,      0,      0,
  0,       0,      1,      0
), nrow = 4, byrow = TRUE)

B <- matrix(c(beta1, 0, 0, 0), nrow = 4, byrow = TRUE)

C <- matrix(c(1, 0, 1, 0), nrow = 1)  # y_t depends on y_t and w_t in state
D <- beta0                            # intercept term

cat("A matrix:\n"); print(A)
cat("B matrix:\n"); print(B)
cat("C matrix:\n"); print(C)
cat("D (intercept):\n"); print(D)

set.seed(123)
n  <- 200
x  <- rnorm(n)
w  <- rnorm(n, sd = 1)

#3. Simulation from original ARMA
y <- numeric(n)
```

```r

# Initialize with zeros
y[1:2] <- 0
w_lag1 <- 0
w_lag2 <- 0

for (t in 3:n) {
  y[t] <- beta0 +
    beta1 * x[t - 1] +
    alpha1 * y[t - 1] +
    alpha2 * y[t - 2] +
    w[t] +
    theta1 * w[t - 1] +
    theta2 * w[t - 2]
}

#4. Simulation in state-space form
s <- matrix(0, nrow = 4, ncol = n)
y_ss <- numeric(n)

for (t in 2:n) {
  noise_vec <- c(0, 0, w[t], 0)
  s[, t] <- A %*% s[, t - 1] + B * x[t - 1] + noise_vec
  y_ss[t] <- as.numeric(C %*% s[, t] + D + w[t])
}

#5. Stability for AR part
# AR polynomial: z^2 - alpha1 * z - alpha2 = 0
roots <- polyroot(c(1, -alpha1, -alpha2))
cat("\nAR(2) roots:\n"); print(roots)
cat("Moduli of roots:\n"); print(Mod(roots))

if (all(Mod(roots) > 1)) {
  cat("\nThe AR(2) part is STABLE\n")
} else {
  cat("\nThe AR(2) part is UNSTABLE\n")
}

#Plot
par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

plot(y, type = "l", col = "blue",
     main = "Simulated yt from ARMA",
     xlab = "t", ylab = "y_t")

plot(y_ss, type = "l", col = "red",
     main = "Simulated yt from state space form",
     xlab = "t", ylab = "y_t")

par(mfrow = c(1, 1))

```

R Code

```r
# ARMAX Assignment: S&P 500 (Output) & Nasdaq (Input)
#  3: Load data and compute log returns

dat <- read.csv("/Users/manishsm/Desktop/Spring 2026/Time Series Analysis/Assignments/quantmod_dataset.csv", stringsAsFactors = FALSE)
dat$Date <- as.Date(dat$Date)
dat <- dat[order(dat$Date), ]

# Log returns of Adjusted Close
# Output (Y): S&P 500 log returns
# Input  (X): Nasdaq log returns
sp500_adj  <- dat$GSPC.Adjusted
nasdaq_adj <- dat$IXIC.Adjusted

sp500_ret  <- diff(log(sp500_adj))   # length N-1
nasdaq_ret <- diff(log(nasdaq_adj))

dates_ret  <- dat$Date[-1]

cat(" Data Summary \n")
cat("Observations (log returns):", length(sp500_ret), "\n")
cat("Date range:", format(min(dates_ret)), "to", format(max(dates_ret)), "\n\n")

# Quick plot of log returns
par(mfrow = c(2,1), mar = c(3,4,2,1))
plot(dates_ret, sp500_ret,  type = "l", col = "blue",
     main = "S&P 500 Log Returns (Output Y)",  ylab = "Log Return", xlab = "")
plot(dates_ret, nasdaq_ret, type = "l", col = "orange",
     main = "Nasdaq Log Returns (Input X)", ylab = "Log Return", xlab = "Date")


# ACF/PACF to guide AR/MA order selection
par(mfrow = c(2,2))
acf(sp500_ret,  main = "ACF  - S&P 500 Returns",  lag.max = 30)
pacf(sp500_ret, main = "PACF - S&P 500 Returns",  lag.max = 30)
acf(nasdaq_ret,  main = "ACF  - Nasdaq Returns",   lag.max = 30)
pacf(nasdaq_ret, main = "PACF - Nasdaq Returns",   lag.max = 30)
par(mfrow = c(1,1))



#  4 & 5: Fit ARMAX models and compare with AIC, BIC, Chi-square (nested)
#
# We use Arima() from {forecast} with xreg = nasdaq_ret (the exogenous input).
# Models considered:
#   M1: ARMAX(1,0,0) -- AR(1) noise
#   M2: ARMAX(0,0,1) -- MA(1) noise
#   M3: ARMAX(1,0,1) -- ARMA(1,1) noise
```

```r
#   M4: ARMAX(2,0,1) -- ARMA(2,1) noise
#   M5: ARMAX(1,0,2) -- ARMA(1,2) noise
#   M6: ARMAX(2,0,2) -- ARMA(2,2) noise (most general)

X <- matrix(nasdaq_ret, ncol = 1)   # exogenous regressor matrix

fit_armax <- function(p, q, xreg = X) {
  Arima(sp500_ret, order = c(p, 0, q), xreg = xreg, include.mean = TRUE)
}

cat(" Fitting ARMAX Models \n")
M1 <- fit_armax(1, 0)  # AR(1)
M2 <- fit_armax(0, 1)  # MA(1)
M3 <- fit_armax(1, 1)  # ARMA(1,1)
M4 <- fit_armax(2, 1)  # ARMA(2,1)
M5 <- fit_armax(1, 2)  # ARMA(1,2)
M6 <- fit_armax(2, 2)  # ARMA(2,2)

models      <- list(M1 = M1, M2 = M2, M3 = M3, M4 = M4, M5 = M5, M6 = M6)
model_names <- c("ARMAX(1,0)", "ARMAX(0,1)", "ARMAX(1,1)",
                 "ARMAX(2,1)", "ARMAX(1,2)", "ARMAX(2,2)")

# Build comparison table
aic_vals <- sapply(models, AIC)
bic_vals <- sapply(models, BIC)
loglik   <- sapply(models, logLik)
npar     <- sapply(models, function(m) length(coef(m)) + 1)  # +1 for sigma^2

comparison <- data.frame(
  Model    = model_names,
  AIC      = round(aic_vals, 2),
  BIC      = round(bic_vals, 2),
  LogLik   = round(as.numeric(loglik), 2),
  nParams  = npar
)
comparison <- comparison[order(comparison$BIC), ]
cat("\n Model Comparison (sorted by BIC) \n")
print(comparison)

# Chi-square tests for nested pairs
# M1 vs M3 (M1 nested in M3): adding MA(1)
# M2 vs M3 (M2 nested in M3): adding AR(1)
# M3 vs M4 (M3 nested in M4): adding AR(2)
# M3 vs M5 (M3 nested in M5): adding MA(2)
# M4 vs M6 (M4 nested in M6): adding MA(2)
# M5 vs M6 (M5 nested in M6): adding AR(2)

lrt <- function(small, large, label) {
  stat <- 2 * (as.numeric(logLik(large)) - as.numeric(logLik(small)))
  df   <- length(coef(large)) - length(coef(small))
  pval <- pchisq(stat, df = df, lower.tail = FALSE)
  cat(sprintf("  %-20s  chi2=%.3f  df=%d  p=%.4f\n", label, stat, df, pval))
}

```

```r
cat("\n Likelihood Ratio (Chi-square) Tests for Nested Models \n")
lrt(M1, M3, "M1 vs M3 (+MA1)")
lrt(M2, M3, "M2 vs M3 (+AR1)")
lrt(M3, M4, "M3 vs M4 (+AR2)")
lrt(M3, M5, "M3 vs M5 (+MA2)")
lrt(M4, M6, "M4 vs M6 (+MA2)")
lrt(M5, M6, "M5 vs M6 (+AR2)")

#
#  6: Choose best model and justify
#
# Criteria: lowest BIC (penalizes complexity more) + chi-square significance.
# Inspect comparison table above; typically ARMAX(1,1) or ARMAX(1,0) is best.

best_name  <- comparison$Model[1]
cat(paste0("\n Best Model (lowest BIC): ", best_name, " \n"))

# Map best_name back to fitted object
best_model <- models[[which(model_names==best_name)]]
cat("\nBest model summary:\n")
print(summary(best_model))

#
#  Residual checking white noise
#
resid_best <- residuals(best_model)

# Autocovariance / ACF of residuals
par(mfrow = c(2, 2))
acf(resid_best,  main = "ACF of Residuals",  lag.max = 40)
pacf(resid_best, main = "PACF of Residuals", lag.max = 40)

# periodogram
spec.pgram(resid_best, taper = 0, log = "no",
           main = "Periodogram of Residuals (Spectral Analysis)")
abline(h = var(resid_best), col = "red", lty = 2)
legend("topright", legend = "Flat spectrum (white noise level)",
       col = "red", lty = 2, cex = 0.8)

# Histogram of residuals
hist(resid_best, breaks = 50, probability = TRUE,
     main = "Histogram of Residuals", xlab = "Residuals", col = "lightblue")
curve(dnorm(x, mean = mean(resid_best), sd = sd(resid_best)),
      add = TRUE, col = "red", lwd = 2)
par(mfrow = c(1,1))

# white noise tests
cat("\nLjung-Box test on residuals (lag 10):\n")
print(Box.test(resid_best, lag = 10, type = "Ljung-Box", fitdf =
sum(!is.na(coef(best_model)))))

cat("\nLjung-Box test on residuals (lag 20):\n")
print(Box.test(resid_best, lag = 20, type = "Ljung-Box", fitdf =
sum(!is.na(coef(best_model)))))
```

```r

cat("\nShapiro-Wilk normality test (on 5000 random residuals):\n")
set.seed(42)
sw_idx <- sample(length(resid_best), min(5000, length(resid_best)))
print(shapiro.test(resid_best[sw_idx]))


#  model parameters
cf <- coef(best_model)
se <- sqrt(diag(vcov(best_model)))

param_table <- data.frame(
  Parameter = names(cf),
  Estimate  = round(cf, 6),
  Std_Error = round(se, 6),
  t_value   = round(cf / se, 3),
  Signif    = ifelse(abs(cf/se) > 2.576, "***",
                     ifelse(abs(cf/se) > 1.960, "**",
                            ifelse(abs(cf/se) > 1.645, "*", "")))
)
rownames(param_table) <- NULL
print(param_table)


#
#  9: Predictions using hypothetical inputs
# Scenario matrix: what if Nasdaq has various hypothetical 1-day log returns?
hyp_nasdaq <- c(-0.04, -0.02, -0.01, 0.00, 0.01, 0.02, 0.04)
labels      <- c("-4%", "-2%", "-1%", "0%", "+1%", "+2%", "+4%")

# Use the last state of the model to forecast 1- ahead for each scenario
pred_sp500 <- numeric(length(hyp_nasdaq))
for (i in seq_along(hyp_nasdaq)) {
  fc <- forecast(best_model, h = 1,
                 xreg = matrix(hyp_nasdaq[i], nrow = 1))
  pred_sp500[i] <- as.numeric(fc$mean)
}

pred_table <- data.frame(
  Nasdaq_Input   = labels,
  Nasdaq_LogRet  = round(hyp_nasdaq, 4),
  SP500_Predicted = round(pred_sp500, 6),
  SP500_Pct       = paste0(round(pred_sp500 * 100, 3), "%")
)
cat("One--ahead S&P 500 log return predictions:\n")
print(pred_table)

# Plot predictions
par(mar = c(5,5,3,2))
plot(hyp_nasdaq * 100, pred_sp500 * 100,
     type = "b", pch = 16, col = "blue", lwd = 2,
     xlab = "Hypothetical Nasdaq Log Return (%)",
     ylab = "Predicted S&P 500 Log Return (%)",
     main = "ARMAX: Predicted S&P 500 Return vs Hypothetical Nasdaq Input")
```

```r
abline(h = 0, lty = 2, col = "grey")
abline(v = 0, lty = 2, col = "grey")
text(hyp_nasdaq * 100, pred_sp500 * 100,
     labels = pred_table$SP500_Pct, pos = 4, cex = 0.8, col = "red")

```